Test Sizes

Monday, December 13, 2010

by Simon Stewart

What do you call a test that tests your application through its UI? An end-to-end test? A functional test? A system test? A selenium test? I've heard all them, and more. I reckon you have too. Tests running against less of the stack? The same equally frustrating inconsistency. Just what, exactly, is an integration test? A unit test? How do we name these things?

Gah!

It can be hard to persuade your own team to settle on a shared understanding of what each name actually means. The challenge increases when you encounter people from another team or project who are using different terms than you. More (less?) amusingly, you and that other team may be using the same term for different test types. "Oh! *That* kind of integration test?" Two teams separated by a common jargon.

Double gah!

The problem with naming test types is that the names tend to rely on a shared understanding of what a particular phrase means. That leaves plenty of room for fuzzy definitions and confusion. There has to be a better way. Personally, I like what we do here at Google and I thought I'd share that with you.

Googlers like to make decisions based on data, rather than just relying on gut instinct or something that can't be measured and assessed. Over time we've come to agree on a set of data-driven naming conventions for our tests. We call them "Small", "Medium" and "Large" tests. They differ like so:

Feature	**Small**	**Medium**	**Large**
Network access	No	localhost only	Yes
Database	No	Yes	Yes
File system access	No	Yes	Yes
Use external systems	No	Discouraged	Yes
Multiple threads	No	Yes	Yes

Sleep statements	No	Yes	Yes
System properties	No	Yes	Yes
Time limit (seconds)	60	300	900+

Going into the pros and cons of each type of test is a whole other blog entry, but it should be obvious that each type of test fulfills a specific role. It should also be obvious that this doesn't cover every possible type of test that might be run, but it certainly covers most of the major types that a project will run.

A Small test equates neatly to a unit test, a Large test to an end-to-end or system test and a Medium test to tests that ensure that two tiers in an application can communicate properly (often called an integration test).

The major advantage that these test definitions have is that it's possible to get the tests to police these limits. For example, in Java it's easy to install a security manager for use with a test suite (perhaps using @BeforeClass) that is configured for a particular test size and disallows certain activities. Because we use a simple Java annotation to indicate the size of the test (with no annotation meaning it's a Small test as that's the common case), it's a breeze to collect all the tests of a particular size into a test suite.

We place other constraints, which are harder to define, around the tests. These include a requirement that tests can be run in any order (they frequently are!) which in turn means that tests need high isolation --- you can't rely on some other test leaving data behind. That's sometimes inconvenient, but it makes it significantly easier to run our tests in parallel. The end result: we can build test suites easily, and run them consistently and as as fast as possible.

Not "gah!" at all.


Labels: Simon Stewart

12 comments :

Cedric December 13, 2010 at 11:46:00 AM PST

Small tests should be isolated from each other, but this constraint gets in a way of medium and large tests (especially for web testing). Take a look at what users are doing with Selenium + TestNG.

Also, dependencies and high parallelism are not mutually exclusive, it's unfortunate that this rumor is still around. Here is why:

http://beust.com/weblog/2009/11/28/hard-core-multicore-with-testng/

Reply

Unknown December 13, 2010 at 12:17:00 PM PST

Hi, thanks for sharing your thoughts on this!

I was wondering how you are dealing with the issue that Medium or Large tests might want to exercise the system along a "path" (for example, perform step a,b,c then verify), where step b might be dependent on state from step a, and the requirement that they tests should be able to run in parallel? Does the Large tests have a huge "setUp" function to get to the state they need to be in before performing the actual test?

Reply

Joe December 13, 2010 at 12:38:00 PM PST

So I don't think you answered your own question.

What do Googlers (who like to make decisions based on data) call a test that tests your application through its UI?

Reply

Mark Roddy December 13, 2010 at 6:44:00 PM PST

Is the time limit referring to a single test (test fixture) or an entire suite?

Reply

Cedric December 14, 2010 at 10:51:00 AM PST

@helino: this is why Selenium users tend to use TestNG: because it supports test dependencies.

Not only does this save a tremendous amount of setup/teardown time (and

less use of statics) but it also leads to more precise reports (i.e. "1 failed, 99 skipped" instead of "100 failed").

Reply

Christian Baumann December 17, 2010 at 6:44:00 AM PST

I like the fact that you can measure the criteria to decide what kind of test you´re talking about. So there´s no room for any fuzzy/blurry interpreations.

To avoid misunderstandings/ miscommunication because of using test-terminology differently, we decided to rely on the ISTQB-glossary as only valid glossary, and for us this works fine.

I also like the idea of independent tests, that don´t rely on each other. We´re trying to do the same at our company, but it´s often very hard to explain to other people.

Reply

jiangfan shi December 17, 2010 at 9:58:00 AM PST

Can you give an example of using these terms, "network access, database, file system, ...", to test an application? Can you give such example with a procedure with step by step?

In my mind, there is a procedure. For example, you may setup an entry main method, and then in the main method you call method A, B, C, and finally you trigger the network access function, D. After you have such path to trigger D, and then you begin to feed small, medium and large dataset to the main function.

If this is the case, then I think you are talking about an input data generation step during an concrete integration testing over a group of components.

Look forward to hear you back, and best wishes.

Reply

m4 December 29, 2010 at 9:10:00 PM PST

I really like your table and the fact that you guys seem to carefully design the kind of tests you're writing.

Limiting runtime of test suites is a new thought to me but very interesting. After all, test execution also has cost attached in a way that somebody might be "waiting" for the Continuous Integration server to "approve" a check-in.

The issue with all the different names for kinds of tests has not been addressed, right? Using a consistent terminology to classify tests surely helps you guys internally but for that purpose, the more common terms to classify test cases would do, too!?

Reply

Anant Verma January 6, 2011 at 3:41:00 PM PST

Limit of 60 seconds for unit tests sound way too high... how do you manage to write a unit test that can take so long?

Reply

Replies

raj March 21, 2017 at 12:57:00 AM PDT

Packaging tests into S, M and L. It again depends.

Reply

Unknown June 14, 2017 at 2:12:00 PM PDT

What if Small tests works less than 60 seconds in release mode and more than 60 seconds in debug mode?

Reply

Unknown October 24, 2019 at 7:24:00 AM PDT

Hi,

Module, Integration, End2end types of test cases shows level of isolation (!) of particular test! And it does not corresponds to "size" of test in any way.

"A Small test equates neatly to a unit test, a Large test to an end-to-end or system test and a Medium " - here you comparing "green" and "soft" objects...their parameters are of different perspectives lets say.

As previous commenters mentioned this terminology of tests are clearly stated in ISTQB standarts and it make sense to check it and understand real purpose behind original such naming.

Reply

Add comment

New comments are not allowed.